

PYTHON MASTER STUDY BOOK

A Complete Textbook for Programming with Python (Course Code
2501IT42)

VOLUME 2

UNIT II — Python Data Types: Strings, Lists, Tuples, Dictionaries, Sets

This volume continues directly from Volume 1 (Unit I). It is full of visual diagrams, step-by-step traces, and try-it-yourself problems designed to make each data type click quickly and stick permanently.

What's New in This Volume

Based on your feedback, this volume adds extra visual and interactive learning tools on top of the Unit I structure:

- **Index diagrams** — box-style diagrams showing every character/element next to its positive and negative index, so slicing finally feels visual instead of abstract.
- **Venn diagrams** — for set operations (union, intersection, difference, symmetric difference), with a concrete example and the relevant zone highlighted for each operation.
- **Step-by-step state traces** — tables that show exactly how a variable changes after each line of code, so you can 'run the program in your head' and check yourself.
- **Flow diagrams** — simple arrow-chain diagrams for multi-step processes like string formatting or dictionary lookups.
- The same Try Yourself → Hint → Solution problem format, GATE focus boxes, comparison tables, and mock test from Volume 1.

UNIT II

Python Data Types

Learning Objectives (CO2)

- Master string creation, indexing, slicing, and the most-used string methods.
- Work confidently with lists and tuples: creating, changing, slicing, and choosing between them.
- Use dictionaries for fast key-based lookup and frequency counting.
- Use sets for membership testing and set algebra (union, intersection, difference).
- Understand WHY each structure behaves the way it does: mutability, hashability, and time complexity.

Prerequisites

Unit I — especially variables, operators, and loops (for/while), since every data type here is explored together with loops.

Part A — Strings

1. String Creation

A **string** is an ordered sequence of characters, written inside single quotes `'...'`, double quotes `"..."`, or triple quotes `'''...'''` / `"""..."""` for text that spans multiple lines.

```
s1 = 'hello'
s2 = "hello"          # identical to s1 -- quote style does not matter
s3 = '''multi
line
string'''
# use double quotes when the text itself contains a single quote
s4 = "It's fine"
```

2. Indexing and Negative Indexing

Every character in a string has a position number called an **index**. Python indexing starts at 0. Python also supports **negative indexing**, counting from the end, with -1 as the last

character — very useful when you don't know the exact length of the string.

`s = "PYTHON"` -- every character with its index

Value	P	Y	T	H	O	N
Index +	0	1	2	3	4	5
Index -	-6	-5	-4	-3	-2	-1

```
s = "PYTHON"
print(s[0])    # P    (first character)
print(s[-1])   # N    (last character)
print(s[2])    # T
print(s[-2])   # O
```

COMMON MISTAKES

- Using an index that is out of range, e.g. `s[10]` on a 6-character string → `IndexError`.
- Forgetting that indexing starts at 0, so the 3rd character is `s[2]`, not `s[3]`.

3. Slicing

Slicing extracts a sub-part of a string using `s[start:stop:step]`. Just like `range()`, `stop` is never included.

```
s = "PYTHON"
print(s[1:4])    # YTH    -- indices 1,2,3 (4 excluded)
print(s[:3])     # PYT    -- start defaults to 0
print(s[3:])     # HON    -- stop defaults to end of string
print(s[:])      # PYTHON -- a full copy of the string
print(s[::2])    # PTO    -- every 2nd character
# NOHTYP -- reversed string, a very common exam/interview trick
print(s[::-1])
```

COMPETITIVE EXAM / GATE FOCUS

`s[::-1]` is the single most common way to reverse a string in Python and appears constantly in exams, MCQs, and interviews. Also remember: slicing NEVER raises an `IndexError` even if the indices are out of range — it simply returns as much as is available (possibly an empty string). This is different from direct indexing, which does raise `IndexError`.

4. Traversing Strings

```
s = "abc"
# traverse by character (most common / most readable)
for ch in s:
    print(ch)

# traverse by index -- use this when you need the position too
for i in range(len(s)):
    print(i, s[i])
```

5. String Immutability

Strings are **immutable**: once created, a string object can never be changed. Any operation that looks like it "modifies" a string actually builds and returns a brand-new string.

```
s = "hello"
# TypeError: 'str' object does not support item assignment
s[0] = "H"

# correct way: build a NEW string and reassign it to s
s = "H" + s[1:]
print(s)          # Hello
```

Trace: `s = "H" + s[1:]`

Step	Code executed	Variable state after
1	<code>s = "hello"</code>	<code>s</code> -> "hello" (id: X)
2	<code>s[1:]</code> evaluated	produces new string "ello"
3	<code>"H" + "ello"</code> evaluated	produces new string "Hello" (a different object, id: Y)
4	<code>s = ...</code> (reassignment)	<code>s</code> now points to "Hello" (id: Y); the old "hello" object is discarded

MUST REMEMBER

- Strings are immutable: no in-place character change is ever possible.
- `s[::-1]` reverses a string. Slicing never raises `IndexError`; direct indexing can.
- "Modifying" a string always means creating a new string and reassigning the variable name to it.

6. String Operators

Operator	Meaning	Example
<code>+</code>	Concatenation (joins two strings)	<code>"foo" + "bar" -> "foobar"</code>
<code>*</code>	Repetition	<code>"ab" * 3 -> "ababab"</code>
<code>in / not in</code>	Membership test	<code>"py" in "python" -> True</code>
<code>==, !=, <, ></code>	Lexicographic (dictionary-order) comparison	<code>"apple" < "banana" -> True</code>

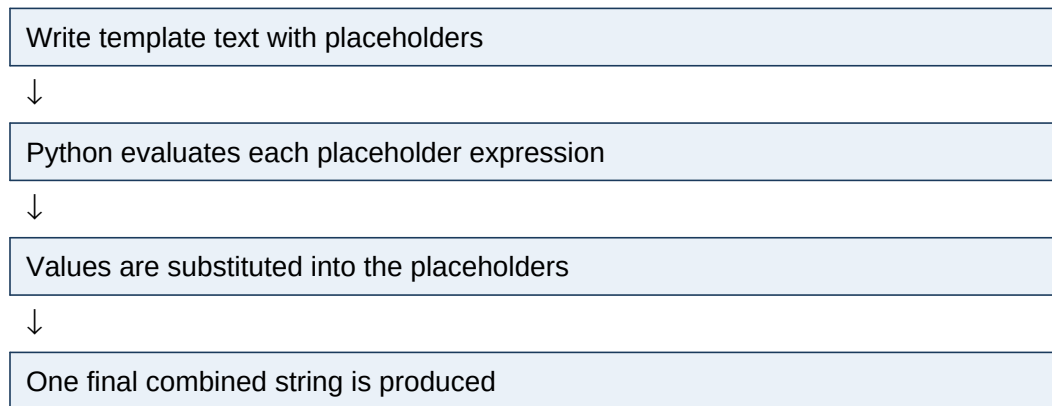
COMPETITIVE EXAM / GATE FOCUS

String comparison is done character by character using each character's Unicode code point (`ord()` value). Uppercase letters (A-Z, 65-90) come BEFORE lowercase letters (a-z, 97-122) in this ordering, so `"Zebra" < "apple"` is `True` — a classic MCQ trap.

7. String Formatting: f-strings and `format()`

Formatting means building a string that includes variable values, nicely arranged.

How string formatting works (conceptually)



```
name = "Asha"
marks = 91.5
```

```
# f-string (Python 3.6+, most modern and most readable -- PREFERRED)
print(f"{name} scored {marks:.1f}%")      # Asha scored 91.5%
```

```
# format() method
print("{} scored {:.1f}%".format(name, marks))
```

```
# % operator (older style, still seen in legacy code and some exams)
print("%s scored %.1f%" % (name, marks))
```

:.1f means "format as a float with 1 digit after the decimal point". Other common format specifiers: :d integer, :, thousands separator, :5d pad to width 5, :05d pad with zeros.

8. String Methods (the ones you will use constantly)

Category	Method	What it does	Example
Case	upper()	Converts to uppercase	"hi".upper() -> "HI"
Case	lower()	Converts to lowercase	"HI".lower() -> "hi"
Case	title()	Capitalizes each word	"hi bob".title() -> "Hi Bob"
Whitespace	strip()	Removes leading/trailing whitespace	" hi ".strip() -> "hi"
Whitespace	lstrip() / rstrip()	Removes only left / only right whitespace	" hi".lstrip()
Search	find(x)	Index of first occurrence, or -1 if absent	"hello".find("l") -> 2

Category	Method	What it does	Example
Search	count(x)	Number of non-overlapping occurrences	"hello".count("l") -> 2
Check	isalpha()	True if all characters are letters	"abc".isalpha() -> True
Check	isdigit()	True if all characters are digits	"123".isdigit() -> True
Check	isalnum()	True if all characters are letters or digits	"abc123".isalnum() -> True
Replace	replace(a,b)	Replaces all occurrences of a with b	"aaa".replace("a","b")
Split	split(sep)	Splits into a list using sep (default: whitespace)	"a,b,c".split(",")
Join	sep.join(list)	Joins list items into one string using sep	"-".join(["a","b"])

```
# split() and join() are opposite operations -- and a very common exam pairing
# ['the', 'quick', 'brown', 'fox']
words = "the quick brown fox".split()
# "the quick brown fox" -- back to original
sentence = " ".join(words)
```

COMMON MISTAKES

- Forgetting that ALL string methods return a NEW string; they never change the original (strings are immutable). `s.upper()` does nothing useful unless you capture the result: `s = s.upper()`.
- Calling `"a,b,,c".split(",")` and being surprised by an empty string in the result — consecutive separators create empty items unless you use `split()` with no arguments (which also collapses multiple whitespace).

INTERVIEW QUESTIONS

Q: How do you check if a string is a palindrome?

A: Compare the string with its reverse: `return s == s[::-1]` (after normalising case/spaces if needed).

Q: How do you count the frequency of each character in a string?

A: Loop through the string and build a dictionary: `for ch in s: freq[ch] = freq.get(ch, 0) + 1`. (`collections.Counter(s)` does the same thing in one line using the standard library.)

Strings — Quick Revision

QUICK REVISION

- Indexing: 0 to `len-1` forward, `-1` to `-len` backward. Slicing `s[start:stop:step]`, `stop` excluded.
- Strings are immutable — every 'change' method returns a new string.
- `s[::-1]` reverses a string. `split()/join()` are inverse operations.
- String comparison is by Unicode code point, so uppercase < lowercase.

String Practice Problems

S1. Check if Substring Present (syllabus: HackerRank find-a-string)

TRY YOURSELF

S1. Check if Substring Present (syllabus: HackerRank find-a-string)

Read a main string and a sub-string. Print whether the sub-string appears in the main string.

Input: main="hello world", sub="wor"

Output: True

HINT

Python's ``in`` operator already checks substring membership in $O(n)$ time on average.

SOLUTION

See code and explanation below.

```
main = input()
sub = input()
print(sub in main)
```

The ``in`` operator on strings performs substring search directly; no manual loop is required. For an interview, be ready to also implement it manually with a sliding window comparing `sub.length` characters at each starting position.

S2. Reverse Vowels of a String (syllabus: LeetCode reverse-vowels-of-a-string)

TRY YOURSELF**S2. Reverse Vowels of a String (syllabus: LeetCode reverse-vowels-of-a-string)**

Reverse only the vowels of a string, keeping all other characters in their original positions.

Input: "hello"

Output: "holle"

HINT

Use two pointers, one from the start and one from the end. Move both inward, and whenever both point to vowels, swap them.

SOLUTION

See code and explanation below.

```
def reverse_vowels(s):
    vowels = set("aeiouAEIOU")
    # strings are immutable, so work with a list
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
        if chars[left] not in vowels:
            left += 1
        elif chars[right] not in vowels:
            right -= 1
        else:
            chars[left], chars[right] = chars[right], chars[left]
            left += 1
            right -= 1
    return "".join(chars)

print(reverse_vowels("hello"))
```

This is the classic **two-pointer technique**: instead of nested loops ($O(n^2)$), two pointers moving toward each other solve it in a single $O(n)$ pass. We convert to a list first because strings cannot be modified in place. Time complexity $O(n)$, space $O(n)$ for the character

list.

S3. Count Vowels/Consonants Variant — Longest Word

TRY YOURSELF

S3. Count Vowels/Consonants Variant — Longest Word

Original practice problem: given a sentence, find and print the longest word (using `split()`).

Input: "the quick brown fox jumps"

Output: quick (or jumps, whichever appears first with max length)

HINT

Split the sentence into words, then use a running-max pattern while looping through the word list.

SOLUTION

See code and explanation below.

```
sentence = input()
words = sentence.split()
longest = words[0]
for w in words:
    if len(w) > len(longest):
        longest = w
print(longest)
```

This combines `split()` with the running-maximum pattern from Unit I: start by assuming the first word is the longest, then update whenever a longer word is found. $O(n)$ time where n = total characters.

Part B — Lists

A **list** is an ordered, **mutable** collection that can hold items of any type, including mixed types and other lists. Lists are written with square brackets.

```
nums = [10, 20, 30, 40]
mixed = [1, "two", 3.0, [4, 5]]    # lists can mix types, even nest
```

Indexing, Slicing, Traversal

```
nums = [10, 20, 30, 40, 50]
```

Value	10	20	30	40	50
Index +	0	1	2	3	4
Index -	-5	-4	-3	-2	-1

Indexing and slicing on lists work exactly like strings, EXCEPT slicing a list returns a new list (not a string), and lists CAN be modified through indexing.

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])    # [20, 30, 40]
print(nums[::-1])   # [50, 40, 30, 20, 10]
nums[0] = 99        # allowed! lists are mutable
print(nums)         # [99, 20, 30, 40, 50]
```

Common List Methods (with before/after state)

Trace: building and modifying a list step by step

Step	Code executed	Variable state after
1	<code>fruits = ['apple','banana']</code>	<code>['apple', 'banana']</code>
2	<code>fruits.append('cherry')</code>	<code>['apple', 'banana', 'cherry']</code> (adds ONE item at the end)
3	<code>fruits.extend(['fig','grape'])</code>	<code>['apple', 'banana', 'cherry', 'fig', 'grape']</code> (adds EACH item of the argument)
4	<code>fruits.insert(1, 'kiwi')</code>	<code>['apple', 'kiwi', 'banana', 'cherry', 'fig', 'grape']</code> (inserts at index 1)
5	<code>fruits.remove('banana')</code>	<code>['apple', 'kiwi', 'cherry', 'fig', 'grape']</code> (removes FIRST match by value)
6	<code>fruits.pop()</code>	<code>['apple', 'kiwi', 'cherry', 'fig']</code> (removes & returns LAST item: 'grape')
7	<code>fruits.sort()</code>	<code>['apple', 'cherry', 'fig', 'kiwi']</code> (sorts in place, alphabetically)
8	<code>fruits.reverse()</code>	<code>['kiwi', 'fig', 'cherry', 'apple']</code> (reverses in place)

Method	Effect	Returns
<code>append(x)</code>	Adds x as a single item at the end	None
<code>extend(iter)</code>	Adds every item of iter individually at the end	None
<code>insert(i,x)</code>	Inserts x at index i	None
<code>remove(x)</code>	Removes the first occurrence of value x	None (ValueError if x not found)
<code>pop(i)</code>	Removes and returns item at index i (default: last item)	the removed item
<code>clear()</code>	Removes all items	None
<code>sort()</code>	Sorts the list in place, permanently changing it	None
<code>sorted(list)</code>	Returns a NEW sorted list, original unchanged	new list
<code>reverse()</code>	Reverses the list in place	None

COMPETITIVE EXAM / GATE FOCUS

`append([1, 2])` adds the whole list `[1,2]` as ONE nested element, so the outer list's length increases by 1. `extend([1,2])` adds 1 and 2 as two separate elements, so length increases by 2. This is one of the most frequently tested output-prediction questions on lists.

```
a = [1, 2]
a.append([3, 4])
print(a)          # [1, 2, [3, 4]]    -- length 3
```

```
b = [1, 2]
b.extend([3, 4])
print(b)          # [1, 2, 3, 4]      -- length 4
```

Copying Lists (a classic pitfall)

```
a = [1, 2, 3]
# WRONG if you wanted a copy -- b is the SAME list object
b = a
# correct shallow copy -- a NEW list with the same top-level items
c = a.copy()
d = a[:]          # another way to shallow copy

b.append(4)
print(a)          # [1, 2, 3, 4]    -- a changed too, because b IS a
c.append(5)
# [1, 2, 3, 4]    -- a is unaffected, because c is a separate list
print(a)
```

COMMON MISTAKES

- Writing `b = a` expecting an independent copy of a list.
- Using `.remove(x)` when you meant to remove BY INDEX — `remove()` searches by VALUE and removes only the first match.
- Calling `.sort()` and then trying to use its return value — it returns `None`; the list is sorted in place.

Nested Lists and List Comprehensions

```
matrix = [[1, 2, 3], [4, 5, 6]]
```

```
print(matrix[1][2])      # 6  -- row index 1, column index 2
```

List comprehension: a compact way to build a list from a loop + optional condition

```
squares = [x * x for x in range(1, 6)]      # [1, 4, 9, 16, 25]
```

```
evens = [x for x in range(1, 11) if x % 2 == 0] # [2, 4, 6, 8, 10]
```

How a list comprehension is evaluated

[expression for item in iterable if condition]

↓

Python loops over the iterable

↓

For each item, the condition is checked (if given)

↓

If True, the expression's value is added to the new list

COMPETITIVE EXAM / GATE FOCUS

A list comprehension [expr for x in seq if cond] is exactly equivalent to: `result = []; for x in seq: if cond: result.append(expr)`. Being able to convert between the two forms instantly is a very common exam and interview task.

List vs Tuple (preview — tuples covered next)

MUST REMEMBER

- Lists are ordered and mutable: [1,2,3], index with [], slice with [:].
- `append()` adds ONE item (even if it's a list); `extend()` adds each item of an iterable separately.
- `b = a` does NOT copy a list; use `.copy()` or `a[:]` or `list(a)` for an independent shallow copy.
- `sort()` sorts in place and returns None; `sorted(x)` returns a new sorted list, leaving `x` unchanged.

INTERVIEW QUESTIONS

Q: What is the time complexity of list indexing, append, and searching?

A: Indexing `list[i]` is $O(1)$ because Python lists are backed by a contiguous array of references. `append()` is $O(1)$ amortized (occasionally $O(n)$ when the underlying array must grow). Searching with `'in'` or `.index()` is $O(n)$ because it may need to check every element.

Part C — Tuples

A **tuple** is an ordered, **immutable** collection, written with parentheses (or often just commas). Once created, its contents cannot be changed, added to, or removed.

```
t1 = (1, 2, 3)
# parentheses are optional -- the comma is what makes it a tuple
t2 = 1, 2, 3
t3 = (5,)          # a ONE-element tuple needs a trailing comma!
# this is just the int 5 in parentheses, NOT a tuple
not_a_tuple = (5)
```

Indexing and slicing work exactly like lists (tuples are ordered), but none of the mutating list methods (append, remove, sort, etc.) are available.

```
t = (10, 20, 30)
print(t[0])          # 10
print(t[1:])         # (20, 30)
# TypeError: 'tuple' object does not support item assignment
t[0] = 99
```

Packing and Unpacking

```
point = 3, 4          # packing -- values packed into a tuple
# unpacking -- tuple values unpacked into separate variables
x, y = point
print(x, y)           # 3 4

# a=1, rest=[2, 3, 4] -- * collects the remaining items into a list
a, *rest = (1, 2, 3, 4)
first, *middle, last = (1,2,3,4,5)  # first=1, middle=[2,3,4], last=5
```

Tuple Functions and Methods

Function / Method	Meaning
len(t)	Number of elements
max(t) / min(t)	Largest / smallest element
sum(t)	Sum of numeric elements
t.count(x)	How many times x appears
t.index(x)	Index of the first occurrence of x
tuple(iterable)	Converts a list/string/etc. into a tuple

List vs Tuple — When to Use Which

List	Tuple
Mutable -- can change after creation.	Immutable -- cannot change after creation.
Use for data that will change: a growing collection, a to-do list.	Use for fixed data: coordinates (x, y), a database record, RGB colour (r, g, b).
Slightly more memory and slightly slower than a tuple of the same data.	More memory-efficient and slightly faster; also hashable if its elements are hashable, so it CAN be used as a dictionary key or set element (a list cannot).
Has many built-in methods (append, sort, etc.).	Has only count() and index().

COMPETITIVE EXAM / GATE FOCUS

"Why are tuples hashable but lists are not?" — Hashing requires an object's value to never change (otherwise its hash value would become invalid for whatever hash table stored it). Since tuples are immutable, Python can safely compute and cache a hash for them, so tuples can be dictionary keys or set members; lists, being mutable, cannot.

QUICK REVISION

- Tuple = ordered, immutable. (5,) is a 1-element tuple; (5) is just the int 5.
- Packing/unpacking: x, y = 3, 4 and a, *rest = [1,2,3,4] are both very common exam patterns.
- Tuples are hashable (if contents are hashable) and so can be dict keys / set elements; lists cannot.

Part D — Dictionaries

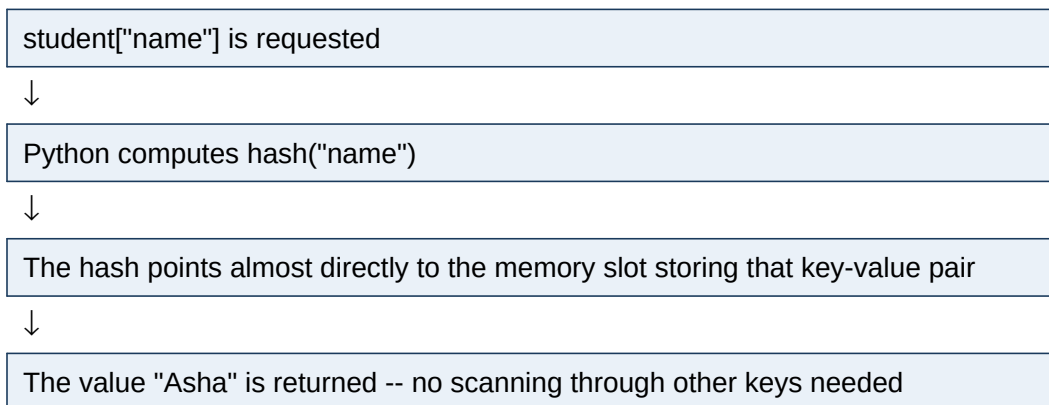
A **dictionary** stores data as **key-value pairs**. Instead of looking up an item by numeric position (like a list), you look it up by a meaningful key. Dictionaries are mutable, and since Python 3.7 they also preserve insertion order.

```
student = {"name": "Asha", "age": 20, "marks": 91.5}
print(student["name"])      # Asha -- lookup by key
# N/A -- safe lookup, no error if key missing
print(student.get("city", "N/A"))
```

Key	Value
"name"	"Asha"
"age"	20
"marks"	91.5

How dictionary lookup works internally

Why dictionary lookup is fast ($O(1)$ average case)



COMPETITIVE EXAM / GATE FOCUS

This hash-based lookup is exactly why dictionary access, insertion, and deletion are $O(1)$ on *average*. It is NOT $O(1)$ in the worst case (many keys could theoretically hash to the same slot, called a collision, degrading performance toward $O(n)$ in pathological cases) — but for exam purposes, 'average case $O(1)$ ' is the expected answer.

Adding, Updating, Removing

Trace: modifying a dictionary

Step	Code executed	Variable state after
1	<code>d = {'a': 1, 'b': 2}</code>	<code>{'a': 1, 'b': 2}</code>
2	<code>d['c'] = 3</code>	<code>{'a': 1, 'b': 2, 'c': 3}</code> (new key -> added)
3	<code>d['a'] = 100</code>	<code>{'a': 100, 'b': 2, 'c': 3}</code> (existing key -> value updated, not duplicated)
4	<code>d.pop('b')</code>	<code>{'a': 100, 'c': 3}</code> (removes key 'b' and returns its value 2)
5	<code>d.update({'d': 4, 'a': 1})</code>	<code>{'a': 1, 'c': 3, 'd': 4}</code> (adds 'd', updates 'a')

Traversing a Dictionary

```
d = {"a": 1, "b": 2, "c": 3}
for key in d:                # loops over KEYS by default
    print(key)

# loops over key-value pairs together -- most useful form
for key, value in d.items():
    print(key, "->", value)

for value in d.values():     # loops over VALUES only
    print(value)
```

Method	Returns
<code>keys()</code>	A view of all keys
<code>values()</code>	A view of all values
<code>items()</code>	A view of (key, value) tuples
<code>get(k, default)</code>	Value for k, or default if k is missing (never raises <code>KeyError</code>)
<code>pop(k)</code>	Removes key k and returns its value (<code>KeyError</code> if missing, unless a default is given)
<code>update(other)</code>	Merges another dictionary in, overwriting matching keys

COMMON MISTAKES

- Using `d["missing_key"]` and getting a `KeyError` — use `d.get("missing_key")` (returns `None`, or your chosen default) when the key might not exist.
- Trying to use a list as a dictionary key → `TypeError`, because lists are unhashable.

Frequency Counting — the #1 real-world use of dictionaries

```
s = "programming"
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
# {'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

`freq.get(ch, 0)` returns the current count for `ch`, or 0 if `ch` has not been seen yet, then we add 1. This pattern ("hash-map counting") is one of the most important patterns for competitive programming and interviews — anagram checks, first-non-repeating character, majority element, and many other problems reduce to this exact pattern.

Nested Dictionaries and Dictionary Comprehensions

```
students = {
    "s1": {"name": "Asha", "marks": 91},
    "s2": {"name": "Ravi", "marks": 78},
}
print(students["s1"]["name"])    # Asha

# {1:1, 2:4, 3:9, 4:16, 5:25}
squares = {x: x*x for x in range(1, 6)}
```

QUICK REVISION

- Dictionary = key-value pairs; average $O(1)$ lookup/insert/delete thanks to hashing.
- `d.get(key, default)` avoids `KeyError`; `d[key]` raises `KeyError` if the key is missing.
- `items()` gives (key, value) pairs together -- the most common way to loop over a dict.
- Frequency counting pattern: `freq[x] = freq.get(x, 0) + 1` inside a loop.
- Only hashable (immutable) types can be dictionary keys: str, int, float, tuple -- never a list.

Part E — Sets

A **set** is an unordered collection of **unique** items — duplicates are automatically removed. Sets are mutable, but every item inside a set must itself be hashable (immutable).

```
s = {1, 2, 3, 2, 1}
print(s)           # {1, 2, 3}  -- duplicates automatically removed

# {} creates an empty DICT, not a set -- this is a classic trap!
empty_set = set()
from_list = set([1, 2, 2, 3])  # {1, 2, 3}
```

COMMON MISTAKES

- Writing `s = {}` expecting an empty set — this actually creates an empty **dictionary**. The correct way to create an empty set is `set()`.

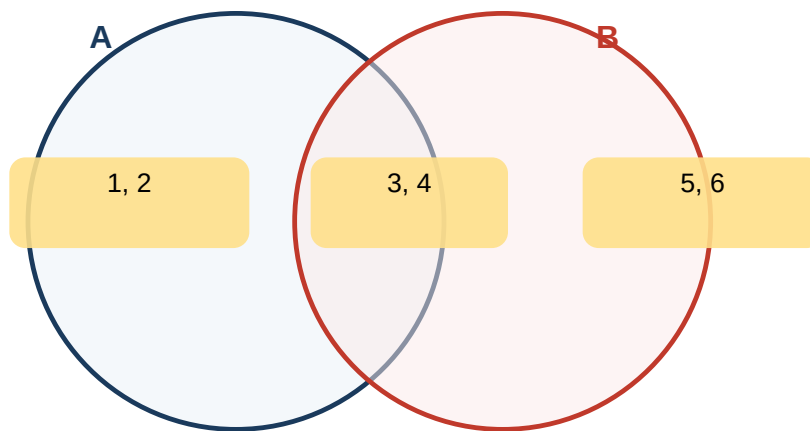
Adding, Removing, Membership

```
s = {1, 2, 3}
s.add(4)           # {1, 2, 3, 4}
s.remove(2)        # {1, 3, 4}  -- KeyError if 2 is not present
s.discard(99)      # no error even though 99 was never in the set
# False -- O(1) average membership test, like dict lookup
print(2 in s)
```

Set Algebra: Union, Intersection, Difference, Symmetric Difference

Example throughout: $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$

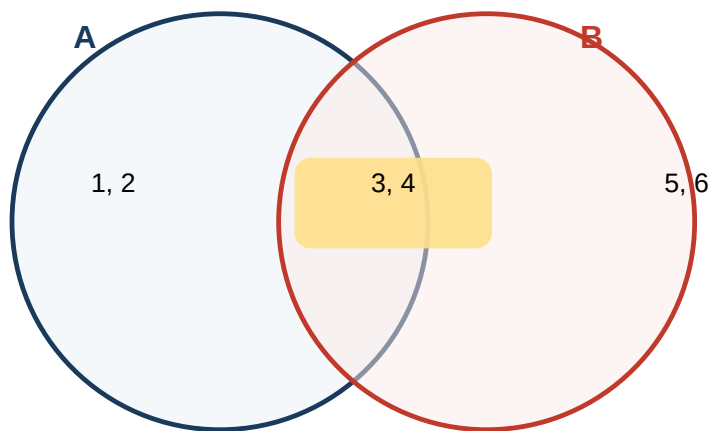
Union ($A \cup B$) — everything in either set



$A \cup B = \{1, 2, 3, 4, 5, 6\}$ (all shaded zones are included)

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)          # {1, 2, 3, 4, 5, 6}
print(A.union(B))     # same result
```

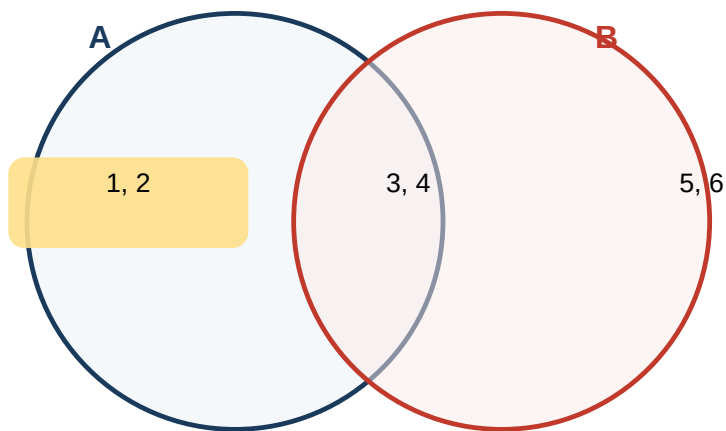
Intersection (A & B) — only what's in both



$A \cap B = \{3, 4\}$ (only the overlap zone is included)

```
print(A & B)          # {3, 4}
print(A.intersection(B)) # same result
```

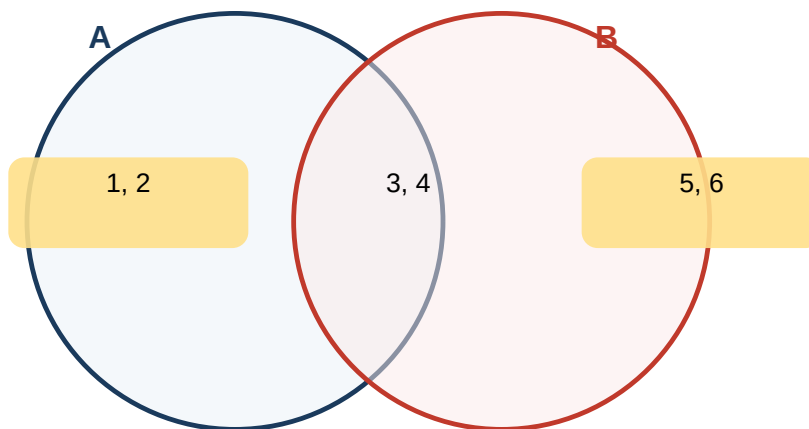
Difference (A - B) — in A but NOT in B



$A - B = \{1, 2\}$ (only A's exclusive zone is included; note $A - B \neq B - A$)

```
print(A - B)          # {1, 2}
print(B - A)          # {5, 6}  -- NOT the same as A - B!
```

Symmetric Difference ($A \oplus B$) — in exactly one of the two sets



$A \oplus B = \{1, 2, 5, 6\}$ (everything EXCEPT the overlap)

```
print(A ^ B)           # {1, 2, 5, 6}
print(A.symmetric_difference(B)) # same result
```

Set Comprehensions

```
evens = {x for x in range(1, 11) if x % 2 == 0}  # {2, 4, 6, 8, 10}
```

Why Sets are so useful in Competitive Programming

COMPETITIVE EXAM / GATE FOCUS

Because membership testing (``in``) is $O(1)$ average in a set versus $O(n)$ in a list, sets are the standard tool for: removing duplicates from a collection in one line (`list(set(items))`), checking 'have I seen this before?' while scanning a list (e.g. detecting the first repeated element), and fast pairwise existence checks (e.g. Two Sum using a set instead of nested loops, turning an $O(n^2)$ brute force into $O(n)$).

Sets — Quick Revision**QUICK REVISION**

- `set()` creates an empty set; `{}` creates an empty **DICTIONARY**.
- Sets auto-remove duplicates and are unordered -- never rely on set item order.
- `|` union, `&` intersection, `-` difference (order matters!), `^` symmetric difference.
- Membership test `'in'` is $O(1)$ average for sets, vs $O(n)$ for lists -- huge for competitive programming.

Choosing the Right Data Structure

Mutable vs Immutable — Full Picture

Type	Mutable?	Hashable? (usable as dict key / set item)
int, float, bool	No	Yes
str	No	Yes
tuple	No	Yes, only if every element inside it is also hashable
list	Yes	No
dict	Yes	No
set	Yes	No
frozenset	No	Yes

Time Complexity of Common Operations

Structure	Access by index/key	Search (in)	Insert	Delete
list	$O(1)$	$O(n)$	$O(1)$ amortized at end / $O(n)$ elsewhere	$O(n)$
tuple	$O(1)$	$O(n)$	N/A (immutable)	N/A (immutable)
dict	$O(1)$ average	$O(1)$ average (checking keys)	$O(1)$ average	$O(1)$ average
set	N/A (no indexing)	$O(1)$ average	$O(1)$ average	$O(1)$ average

Which structure should I use?

Need	Use
Ordered collection that may change (add/remove/reorder)	list
Fixed collection that should never change (a record, coordinates)	tuple
Fast lookup by a meaningful key / label; frequency counting	dict

Need	Use
Uniqueness required; fast membership testing; set algebra	set

Master Comparison Tables (Unit II)

List vs Tuple

List	Tuple
[1,2,3] -- mutable	(1,2,3) -- immutable
Slower, more memory	Faster, less memory
Not hashable	Hashable if contents are hashable

List vs Set

List	Set
Ordered, allows duplicates	Unordered, duplicates removed automatically
Search is $O(n)$	Search is $O(1)$ average
Supports indexing <code>s[i]</code>	No indexing at all

Set vs Dictionary

Set	Dictionary
Stores only values: {1,2,3}	Stores key-value pairs: {"a":1}
No concept of lookup by key	Designed exactly for fast lookup by key
Written {v1, v2}	Written {k1:v1, k2:v2}

append() vs extend()

append(x)	extend(iter)
Adds x as ONE new item, whatever x is	Adds EACH element of iter as a separate item
List length increases by exactly 1	List length increases by len(iter)

remove() vs pop()

remove(x)	pop(i)
Removes by VALUE (first match)	Removes by INDEX (default: last item)
Returns None	Returns the removed item
ValueError if x not found	IndexError if i out of range

sort() vs sorted()

sort()	sorted(x)
A LIST METHOD -- sorts in place, changes the original list	A BUILT-IN FUNCTION -- returns a NEW sorted list/collection
Returns None	Returns the new sorted list
Only works on lists	Works on any iterable (list, tuple, string, dict keys...)

More Practice Problems (Lists, Dicts, Sets)

B. Easy

L1. Generate 20 Random Numbers: Min, Max, Avg, Even Count (syllabus: CodeChef FLOW002)

TRY YOURSELF

L1. Generate 20 Random Numbers: Min, Max, Avg, Even Count (syllabus: CodeChef FLOW002)

Generate 20 random integers between 1 and 100. Print the minimum, maximum, average, and how many are even.

Input: (randomly generated)

Output: Min: .. Max: .. Avg: .. Even count: ..

HINT

Use the random module to build the list, then use built-in min(), max(), sum(), and a loop for the even count.

SOLUTION

See code and explanation below.

```
import random

nums = [random.randint(1, 100) for _ in range(20)]
print("Numbers:", nums)
print("Min:", min(nums))
print("Max:", max(nums))
print("Avg:", sum(nums) / len(nums))
even_count = sum(1 for n in nums if n % 2 == 0)
print("Even count:", even_count)
```

A list comprehension with random.randint() generates the 20 numbers. min(), max() and sum() are built-in functions that scan the whole list in O(n). The even count uses a generator expression inside sum(): each even number contributes 1, giving the total

count. Overall $O(n)$ time.

L2. Product of Three Numbers (syllabus: LeetCode maximum-product-of-three-numbers)

TRY YOURSELF

L2. Product of Three Numbers (syllabus: LeetCode maximum-product-of-three-numbers)

Practice problem listed in syllabus — given a list of integers, find the maximum product obtainable by multiplying any three of them.

Input: [1, 2, 3, 4]

Output: 24

HINT

Sort the list. The answer is either (largest three) OR (two smallest negative numbers * largest number) -- think about why negative numbers matter.

SOLUTION

See code and explanation below.

```
nums = [1, 2, 3, 4]
nums.sort()
n = len(nums)
option1 = nums[-1] * nums[-2] * nums[-3]          # three largest
# two smallest (possibly negative) * largest
option2 = nums[0] * nums[1] * nums[-1]
print(max(option1, option2))
```

After sorting, the three largest values are always at the end. But two very negative numbers multiply to a large POSITIVE number, so pairing the two smallest with the single largest can sometimes beat using the three largest directly. We compute both candidates and take the max. Sorting costs $O(n \log n)$, which dominates the total time complexity.

C. Medium / Competitive

L3. Longest Consecutive Sequence (syllabus: LeetCode longest-consecutive-sequence)

TRY YOURSELF

L3. Longest Consecutive Sequence (syllabus: LeetCode longest-consecutive-sequence)

Practice problem listed in syllabus — given an unsorted list of integers, find the length of the longest run of consecutive integers (they do not need to be consecutive in the list, just consecutive in value).

Input: [100, 4, 200, 1, 3, 2]

Output: 4 (the sequence 1,2,3,4)

HINT

Put all numbers in a set for O(1) lookup. Only START counting a sequence from a number that has no predecessor (num-1) in the set -- this avoids re-counting the same run many times.

SOLUTION

See code and explanation below.

```
def longest_consecutive(nums):
    num_set = set(nums)
    best = 0
    for num in num_set:
        # num is the START of a sequence
        if num - 1 not in num_set:
            length = 1
            current = num
            while current + 1 in num_set:
                current += 1
                length += 1
            best = max(best, length)
    return best

print(longest_consecutive([100, 4, 200, 1, 3, 2]))
```

Converting to a set gives $O(1)$ membership checks. The key trick: we only begin expanding a sequence when the current number has no left-neighbour in the set, which guarantees each number is the starting point of at most one expansion. This keeps total work $O(n)$ overall instead of $O(n^2)$, even though there is a while loop inside a for loop.

L4. Subarray Sum Equals K (syllabus: LeetCode subarray-sum-equals-k)

TRY YOURSELF

L4. Subarray Sum Equals K (syllabus: LeetCode subarray-sum-equals-k)

Practice problem listed in syllabus — given a list of integers and a target k , count how many contiguous subarrays sum exactly to k .

Input: `nums=[1,1,1], k=2`

Output: 2

HINT

Use a running prefix sum and a dictionary that counts how many times each prefix sum has occurred. If $(\text{current_sum} - k)$ has occurred before, those earlier positions mark valid subarrays ending here.

SOLUTION

See code and explanation below.

```
def subarray_sum(nums, k):
    count = 0
    prefix_sum = 0
    seen = {0: 1}          # empty prefix (sum 0) has occurred once
    for num in nums:
        prefix_sum += num
        if (prefix_sum - k) in seen:
            count += seen[prefix_sum - k]
        seen[prefix_sum] = seen.get(prefix_sum, 0) + 1
    return count

print(subarray_sum([1, 1, 1], 2))
```

This is the classic **prefix-sum + hash-map** pattern. If the running sum up to index j minus the running sum up to some earlier index i equals k , then the subarray between $i+1$ and j sums to k . Instead of checking every possible subarray ($O(n^2)$), we store how many times each prefix sum has appeared and look it up in $O(1)$, bringing total time down to $O(n)$.

D. Interview-style

L5. String Matching in an Array (syllabus: LeetCode string-matching-in-an-array)

TRY YOURSELF

L5. String Matching in an Array (syllabus: LeetCode string-matching-in-an-array)

Practice problem listed in syllabus — given a list of words, return every word that is a substring of at least one other word in the list.

Input: ["mass","as","hero","superhero"]

Output: ["as", "hero"]

HINT

For each word, check if it is a substring of any OTHER word in the list using the ``in`` operator.

SOLUTION

See code and explanation below.

```
def string_matching(words):
    result = []
    for i, w in enumerate(words):
        for j, other in enumerate(words):
            if i != j and w in other:
                result.append(w)
                break
    return result
```

```
print(string_matching(["mass", "as", "hero", "superhero"]))
```

Two nested loops compare every word against every other word; `in` performs the substring check. We break out of the inner loop as soon as one match is found, to avoid adding the same word twice. Time complexity is roughly $O(n^2 \cdot m)$ where n is the number of words and m is average word length (because each 'in' check itself takes time proportional to string length).

Unit II Mock Test

1. What is the output of: `print("Python"[-2])`?
(a) o (b) n (c) h (d) O
2. What is the output of: `print([1,2,3] + [4,5])`?
(a) [1,2,3,4,5] (b) Error (c) [5,7,3] (d) [[1,2,3],[4,5]]
3. Which creates an empty set?
(a) {} (b) set() (c) [] (d) set[]
4. `a=[1,2]; b=a; b.append(3); print(a)` ->
(a) [1,2] (b) [1,2,3] (c) Error (d) None
5. What is (5) vs (5,) ?
(a) both tuples (b) (5) is int, (5,) is tuple (c) both ints (d) Error
6. `d={'a':1}; print(d.get('b', 0))` ->
(a) KeyError (b) None (c) 0 (d) 'b'
7. What is `A - B` for `A={1,2,3}`, `B={2,3,4}`?
(a) {1} (b) {4} (c) {1,4} (d) {2,3}
8. Which has $O(1)$ average membership testing?
(a) list (b) tuple (c) set (d) string
9. `sorted([3,1,2])` vs `[3,1,2].sort()` -- which is True?
(a) both return a new list (b) sorted returns new list, .sort() returns None (c) both return None (d) .sort() returns a new list
10. What does `"abc".upper()` return if `s` was never reassigned?
(a) modifies `s` to ABC (b) returns "ABC" but `s` is unchanged unless reassigned (c) Error (d) None

Coding Questions

- Write a program to check if a given string is a palindrome, ignoring case.
- Write a program to remove duplicate elements from a list while preserving order.
- Write a program that counts the frequency of each word in a sentence using a dictionary.
- Write a program using sets to find common elements between two lists.

- Trace by hand: `d={}`; for `ch` in `"aab"`: `d[ch]=d.get(ch,0)+1`; `print(d)` -- what is printed?

Answer Key

Q	Answer	Q	Answer
1	(b) n -- second-last character	6	(c) 0 -- default value returned, key 'b' missing
2	(a) [1,2,3,4,5] -- list + list concatenates	7	(a) {1} -- elements only in A
3	(b) set() -- {} makes an empty dict	8	(c) set -- hash-based O(1) average
4	(b) [1,2,3] -- b is the same object as a	9	(b) sorted returns new list; .sort() returns None
5	(b) (5) is just int 5; (5,) is a 1-tuple	10	(b) returns "ABC"; s stays lowercase unless reassigned

Coding question 5 trace: `d` starts `{}`. `ch='a': d={'a':1}`. `ch='a': d={'a':2}`. `ch='b': d={'a':2,'b':1}`.
 Final output: **`{'a': 2, 'b': 1}`**.

End of Unit II. Unit III (Functions and Packages, including recursion) will follow next, with the same visual and interactive structure.